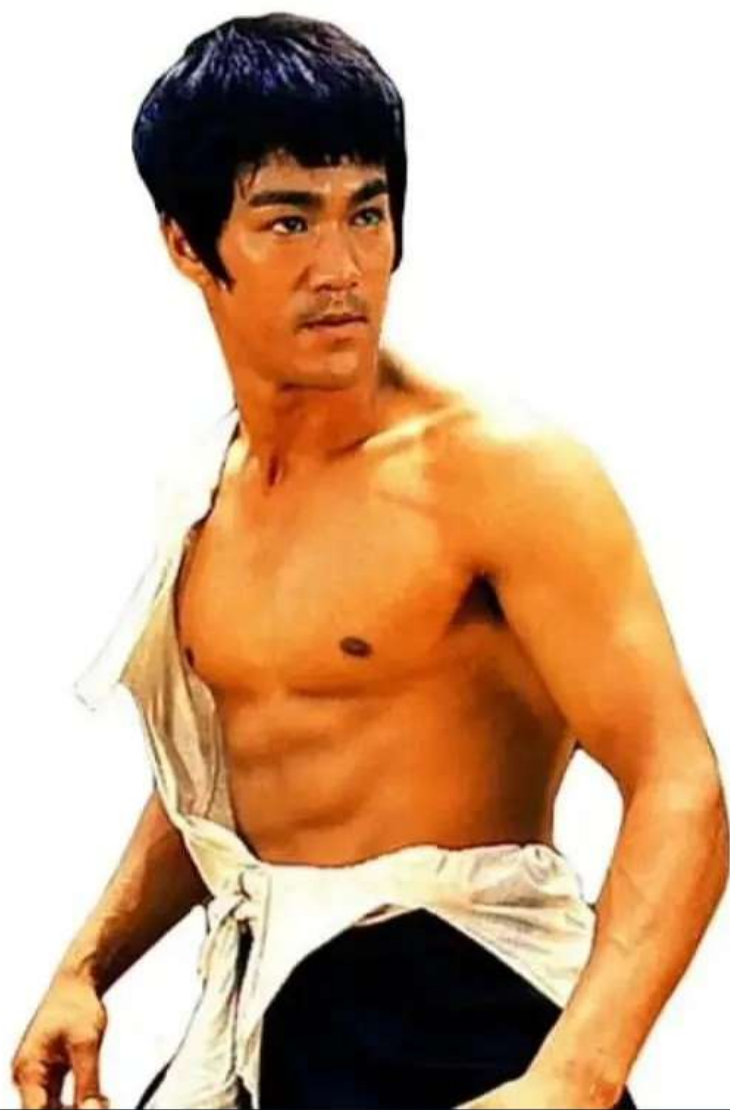
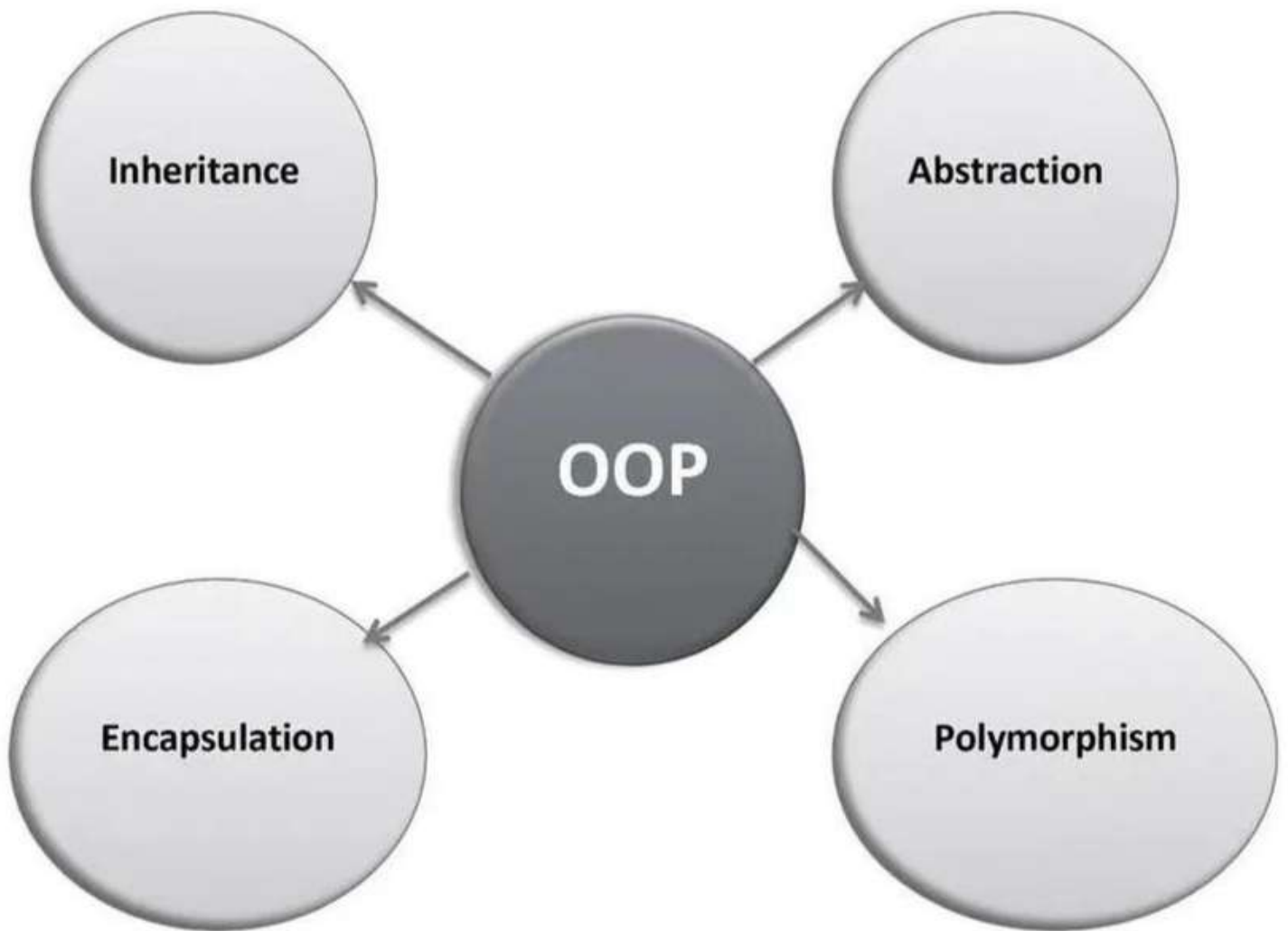


OOPS

In Real Life





Inheritance



**The Girls can have same color
same size, same name.
But they are not same, they are
similar.**

Encapsulation



A college can have many departments like CS Department, Accounts Department, etc.

All these departments together makes a college.

Polymorphism



Like he is an actor,
YouTuber, doctor,
son, brother etc.

A man can have many roles like
actor, doctor, Astronaut, son
etc.

Same person can have different
roles.

Abstraction



With a phone you can do many things like phone calls, messages, play games etc.

It doesn't show the inside process of how its doing the things. implementation parts are hidden.

1. Classes And Objects

- **Classes** are blueprints for objects. They define the properties and behaviors that objects of the class will have.
- **Objects** are instances of classes. They represent real-world entities and have states and behaviors.
- Example:

```
class Car {  
    String brand;  
    int year;  
    void start() {  
        // Code to start the car  
    }  
}  
  
Car myCar = new Car();  
myCar.brand = "Toyota";  
myCar.year = 2022;  
myCar.start();
```

2. Inheritance

- Inheritance allows a class (subclass/derived class) to inherit properties and behaviors from another class (superclass/base class).
- It promotes code reusability and establishes an "is-a" relationship.
- Example:

```
class Animal {  
    void eat() {  
        System.out.println("Animal is eating...");  
    }  
}  
  
class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog is barking...");  
    }  
}  
  
Dog myDog = new Dog();  
myDog.eat(); // Inherited from Animal  
myDog.bark();
```

3. Polymorphism

- Polymorphism allows objects of different classes to be treated as objects of a common superclass.
- It can be achieved through method overriding (runtime polymorphism) and method overloading (compile-time polymorphism).
- Example (Method Overriding):

```
class Animal {  
    void makeSound() {  
        System.out.println("Animal is making a sound...");  
    }  
}  
  
class Dog extends Animal {  
    @Override  
    void makeSound() {  
        System.out.println("Dog is barking...");  
    }  
}  
  
Animal myAnimal = new Dog();  
myAnimal.makeSound(); // Calls Dog's makeSound() method
```

4. Encapsulation

- Encapsulation is the bundling of data (attributes) and methods that operate on the data into a single unit (class).
- It restricts direct access to data and mandates access through methods (getters and setters).
- Example:

```
class Student {  
    private String name;  
    private int age;  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
  
    // Similar getter and setter for age  
}
```

5. Abstraction

- Abstraction focuses on hiding the implementation details and showing only the essential features of an object.
- Abstract classes and interfaces are used to achieve abstraction.
- Example:



```
abstract class Shape {  
    abstract void draw(); // Abstract method  
}  
  
class Circle extends Shape {  
    void draw() {  
        System.out.println("Drawing a circle...");  
    }  
}
```

6. Constructors

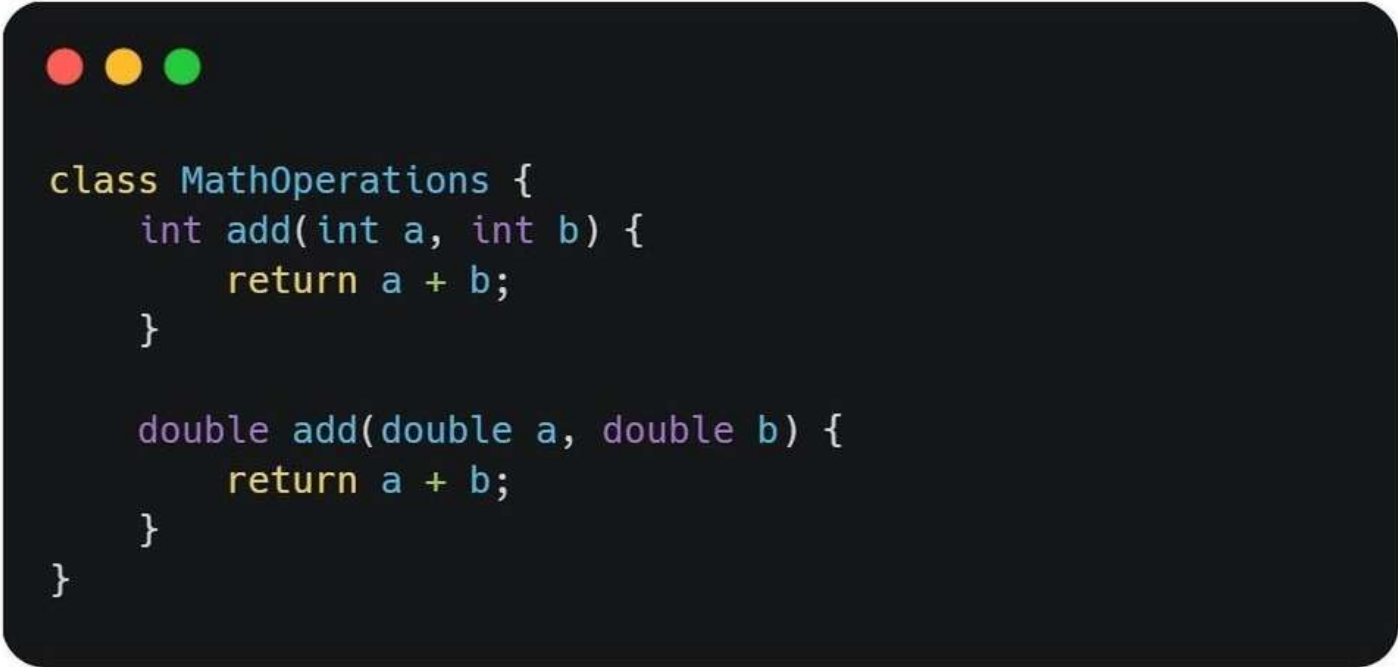
- Constructors are special methods used to initialize objects. They have the same name as the class and no return type.
- They are called automatically when an object is created.
- Example:



```
class Car {  
    String brand;  
  
    // Constructor  
    public Car(String brand) {  
        this.brand = brand;  
    }  
}  
  
Car myCar = new Car("Toyota");
```

7. Overloading And Overriding

- Overloading occurs when a class has multiple methods with the same name but different parameters.
- Overriding occurs when a subclass provides a specific implementation for a method that is already defined in its superclass.
- Example (Overloading):



```
class MathOperations {  
    int add(int a, int b) {  
        return a + b;  
    }  
  
    double add(double a, double b) {  
        return a + b;  
    }  
}
```

- Example (Overriding):



```
class Animal {  
    void makeSound() {  
        System.out.println("Animal is making a sound...");  
    }  
}  
  
class Dog extends Animal {  
    @Override  
    void makeSound() {  
        System.out.println("Dog is barking...");  
    }  
}
```